Computer Vision Engineer — Technical Assessment

Task Description

Build a **production-ready visual defect detection system** for the following scenario:

A manufacturing client captures images of products on a production line. Some products contain visible defects, while others are normal. Today, these products are inspected manually.

Your task is to build a computer vision system that can automatically determine whether a product is **normal or defective**.

You will be provided with a small dataset containing normal and defective product images.

The solution should:

- Analyze and prepare the provided dataset.
- Build a model that classifies products as:
    - Normal
    - Defective
- Use appropriate preprocessing and image augmentation.
- Handle class imbalance where necessary.
- Split the dataset appropriately into training, validation, and test sets.
- Select an appropriate model architecture and explain why you selected it.
- Use transfer learning where appropriate.
- Evaluate the model using appropriate metrics such as:
    - Precision
    - Recall
    - F1-score
    - Confusion matrix
- Perform basic error analysis on false positives and false negatives.
- Expose the trained model through an API where an image can be submitted for inference.
- Return:
    - Predicted class
    - Confidence score
- Include proper validation, logging, and error handling.
- Package the application so it can realistically be deployed and maintained.

You may use **Python, PyTorch, TensorFlow, OpenCV, scikit-learn, Docker, ONNX, cloud services, or any other appropriate tools**.

We are not looking for a specific model or framework. We want to understand **how you approach the dataset, why you selected your model and evaluation strategy, how you analyze failures, and whether you can turn a trained model into a production-ready service.**

Evaluation

Your submission will be evaluated based on:

- Dataset analysis and preparation
- Model selection and reasoning
- Training and evaluation approach
- Handling of class imbalance
- Understanding of false positives and false negatives
- Error analysis and identification of model weaknesses
- Code quality and project structure
- Production inference implementation
- Ability to explain accuracy, latency, and model trade-offs

Submission

Please provide:

- GitHub repository with your code
- README containing:
    - Setup instructions
    - Model approach
    - Dataset strategy
    - Evaluation results
    - Known limitations
- Architecture diagram
- Trained model or instructions for reproducing training
- Working inference API
- Example predictions
- Docker setup or deployment instructions
- 2–3 minute video demonstrating the solution and explaining your main technical decisions

Expected effort: approximately 6–8 hours.
Submission window: 24 hours.

Use of AI tools is allowed. However, you must be able to explain, modify, and debug everything you submit during the technical interview.